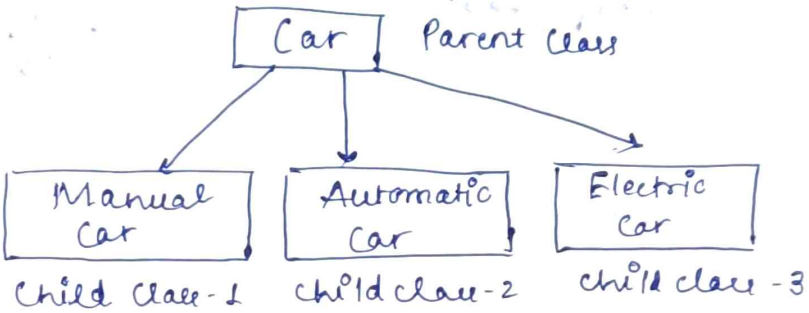
LECTURE-3

INHERITANCE

→ When one class inherits the property of another class this is called inheritance.



⇒ Car – characterstics
- → Brand
- → Model
- → IsEngine On
- → Currentspeed

startengine()
stopengine()
accelerate()
break()

These properties are common for every class.

⇒ Manual Car
Characterstics
- → Current Gear
- → shift Gear

⇒ Electric Car
- → Battery Percentage
- → Charge battery

```
Class Car {
        Attribute 1;
        Attribute 2;
        Behaviour 1;
}
```

This is Inheritance

```
Class Manual Car : public Car {
        In this we can use Attribute 1,
        Attribute 2 and Behaviour 1
        Shift Gear ( ){ }. // and also create
                                specific for this
                                class
}
```

→ we can create more classes like Electric Car similar as manual car.

➡ Class Manual Car : public Car { }

Q- what if write private or protected instead of public.

→ when it is public – After inheriting all the attributes of behaviour and remains same as in main class.

if public then public
if private then private
if protected then protected

→ when it is private – all behaviour and attributes becomes private and if their is a class that inherit manual car it can't use the properties of main class.

→ If it is protected- Can't access private
protected becomes protected
public becomes protected.

POLYMORPHISM

⇒ Poly – many
Morphism - forms

In this different behaviour react different at different stages, that is polymorphism.

⇓

- Static Polymorphism
  - ↓ Method overloading
- Dynamic Polymorphism
  - ↓ Method overriding

⇒ Car
- → brand
- → Model
- → isEngineOn
- → Current Speed
- → accelerate()

In this we say that every vehicle accelerate but rate is diff.

```
Class Car {
virtual accelerate () = 0 ;
}
```

} declaration of accelerate

```
Class ManualCar {
    accelerate()
}
```

```
Class ElectricCar {
    accelerate()
}
```

They defines accelerate() as their own way

Static Polymorphism -

Human

run()
↳ Is in danger run fast() (If True) else run slow()

In this we change parameter and the behaviour is changed.

Eg.

```
Manual_Car {
    accelerate();
    accelerate (int speed);
}
```

Operator Overloading - is when we make an operator like +, -, * etc work differently when used with objects of a class. For example- normally + adds number. But if we overload it for a person class, it could combine their names. If we talk about C++ or Python they both support operator overloading on the other if we talk about Java or JS they didn't support just because of to maintain simplicity, clarity and predictable behaviour, avoiding complexities & error.